

```

{
  "cells": [
    {
      "cell_type": "markdown",
      "id": "f6dd7729-1e2a-4065-9447-086fa839d57a",
      "metadata": {},
      "source": [
        "# 🇵🇰 Customer Churn Prediction - Telecom\n",
        "***Internship Project by Mohsin Khan** \n",
        "_A comprehensive predictive model to identify atâ€risk customers._\n",
        "\n",
        "---\n",
        "\n",
        "## 1: Data Generation\n",
        "Below we run the provided code to generate the mock dataset for 2,000 customers.\n"
      ]
    },
    {
      "cell_type": "code",
      "execution_count": null,
      "id": "9f1a46fb-1d08-4ee9-93ab-23386233547a",
      "metadata": {},
      "outputs": [],
      "source": [
        "import pandas as pd\n",
        "import numpy as np\n",
        "\n",
        "# (Rest of imports if you need seaborn/matplotlib later)\n",
        "import seaborn as sns\n",
        "import matplotlib.pyplot as plt\n"
      ]
    },
    {
      "cell_type": "code",
      "execution_count": null,
      "id": "97b3d987-b26c-4995-a98b-6f63754c2944",
      "metadata": {},
      "outputs": [],
      "source": [
        "# Generate mock customer data\n",
        "np.random.seed(42)\n",
        "num_customers = 2000\n",
        "\n",
        "data = {\n",
        "    'CustomerID': [f'CUST{1000+i}' for i in range(num_customers)],\n",
        "    'Gender': np.random.choice(['Male', 'Female'], num_customers, p=[0.5, 0.5]),\n",
        "    'SeniorCitizen': np.random.choice([0, 1], num_customers, p=[0.84, 0.16]),\n",
        "    'Partner': np.random.choice(['Yes', 'No'], num_customers, p=[0.48, 0.52]),\n",
        "    'Dependents': np.random.choice(['Yes', 'No'], num_customers, p=[0.3, 0.7]),\n",
        "    'Tenure': np.random.randint(1, 73, num_customers), # Months\n",
        "    'PhoneService': np.random.choice(['Yes', 'No'], num_customers, p=[0.9, 0.1]),\n",
        "    'MultipleLines': np.random.choice(\n",
        "        ['Yes', 'No', 'No phone service'], num_customers, p=[0.42, 0.48, 0.1]),\n",
        "    'InternetService': np.random.choice(\n",
        "        ['DSL', 'Fiber optic', 'No'], num_customers, p=[0.34, 0.44, 0.22]),\n",
        "    'OnlineSecurity': np.random.choice(\n",
        "        ['Yes', 'No', 'No internet service'], num_customers, p=[0.28, 0.50, 0.22]),\n",
        "    'OnlineBackup': np.random.choice(\n",
        "        ['Yes', 'No', 'No internet service'], num_customers, p=[0.34, 0.44, 0.22]),\n",
        "    'DeviceProtection': np.random.choice(\n",
        "        ['Yes', 'No', 'No internet service'], num_customers, p=[0.34, 0.44, 0.22]),\n",
        "    'TechSupport': np.random.choice(\n",
        "        ['Yes', 'No', 'No internet service'], num_customers, p=[0.29, 0.49, 0.22]),\n",
        "    'StreamingTV': np.random.choice(\n",
        "        ['Yes', 'No', 'No internet service'], num_customers, p=[0.38, 0.40, 0.22]),\n",
        "    'StreamingMovies': np.random.choice(\n",
        "        ['Yes', 'No', 'No internet service'], num_customers, p=[0.39, 0.39, 0.22]),\n",
        "    'Contract': np.random.choice(\n",
        "        ['Month-to-month', 'One year', 'Two year'], num_customers, p=[0.55, 0.24, 0.21]),\n",
        "    'PaperlessBilling': np.random.choice(['Yes', 'No'], num_customers, p=[0.59, 0.41]),\n",
        "    'PaymentMethod': np.random.choice(\n",
        "        ['Electronic check', 'Mailed check', 'Bank transfer (automatic)', 'Credit card (automatic)'],\n",
        "        num_customers, p=[0.34, 0.23, 0.22, 0.21]),\n",
        "    'MonthlyCharges': np.random.normal(loc=65, scale=30, size=num_customers).clip(18, 120).round(2),\n",
        "    'TotalCharges': np.nan\n",
        "}\n",
        "df_customers = pd.DataFrame(data)\n",
        "\n",
        "# Generate TotalCharges based on Tenure and MonthlyCharges with some noise\n",
        "df_customers['TotalCharges'] = (\n",
        "    df_customers['Tenure']\n",
        "    * df_customers['MonthlyCharges']\n",
        "    * np.random.uniform(0.95, 1.05, num_customers)\n",
        ").round(2)\n",
        "\n",
        "# Make some TotalCharges empty for realism (e.g., new customers with 0 tenure)\n",
        "df_customers.loc[df_customers['Tenure'] == 1, 'TotalCharges'] = df_customers['MonthlyCharges']\n",
        "\n",
        "# **Fix:** select 1% of those with Tenure < 3, then set TotalCharges to NaN\n",
        "tenure_lt_3_idx = df_customers[df_customers['Tenure'] < 3].index\n",
        "random_idx = np.random.choice(tenure_lt_3_idx, size=int(num_customers * 0.01), replace=False)\n",
        "df_customers.loc[random_idx, 'TotalCharges'] = np.nan\n",
        "\n",
        "# Simulate Churn (more likely for month-to-month, higher charges, lower tenure)\n",
        "churn_probability = (\n",
        "    0.1\n",

```

```

"    + 0.15 * (df_customers['Contract'] == 'Month-to-month')\n",
"    + 0.1 * (df_customers['InternetService'] == 'Fiber optic')\n",
"    + 0.001 * (df_customers['MonthlyCharges'] - 65)\n",
"    - 0.002 * (df_customers['Tenure'] - 36)\n",
"    + 0.1 * (df_customers['OnlineSecurity'] == 'No')\n",
"    + 0.1 * (df_customers['TechSupport'] == 'No')\n",
"  )\n",
  churn_probability = np.clip(churn_probability, 0.01, 0.99)\n",
  "\n",
  df_customers['Churn'] = np.random.binomial(1, churn_probability, num_customers).astype(str)\n",
  df_customers['Churn'] = df_customers['Churn'].replace({'1': 'Yes', '0': 'No'})\n",
  "\n",
  "# Replace 'No phone service' and 'No internet service' for consistency\n",
  for col in ['MultipleLines']:\n",
    df_customers[col] = df_customers.apply(\n",
      lambda row: 'No' if row['PhoneService'] == 'No' else row[col], axis=1\n",
    )\n",
  "\n",
  for col in ['OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies']:\n",
    df_customers[col] = df_customers.apply(\n",
      lambda row: 'No' if row['InternetService'] == 'No' else row[col], axis=1\n",
    )\n",
  "\n",
  "\n",
  "# Save to CSV\n",
  df_customers.to_csv('telecom_churn_mock_data.csv', index=False)\n",
  print("\nMock telecom churn data generated: telecom_churn_mock_data.csv")\n",
  print(df_customers.head())\n",
  print(f"\nChurn distribution:\n{df_customers['Churn'].value_counts(normalize=True)}")\n"
]
},
{
  "cell_type": "markdown",
  "id": "0c1ddffc-cb36-47ff-b81c-391b9af60a24",
  "metadata": {},
  "source": [
    "# 2. Display Top 5 Rows Of Dataset\n"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "68eb21d6-b341-4b72-9463-03d78c714af3",
  "metadata": {},
  "outputs": [],
  "source": [
    df_customers.head()\n"
  ]
},
{
  "cell_type": "markdown",
  "id": "75d60aa0-5825-40e7-86a8-51d47fc59d86",
  "metadata": {},
  "source": [
    "# 3. Display Last 5 Rows Of Dataset\n"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "607b24d0-db22-480f-99dd-c7c01670c922",
  "metadata": {},
  "outputs": [],
  "source": [
    df_customers.tail()\n"
  ]
},
{
  "cell_type": "markdown",
  "id": "a13a3b68-7c1a-4be3-bab8-d120b4da2046",
  "metadata": {},
  "source": [
    "# 4. Find shape of our dataset (number of rows,number of columns)\n",
    "\n",
    "\n"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "ed3abdd3-70a3-42fa-9cdd-8e1e660cde2f",
  "metadata": {},
  "outputs": [],
  "source": [
    df_customers.shape\n"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "a15e6552-c430-4b8c-8d72-7030466b1db2",
  "metadata": {},
  "outputs": [],
  "source": [
    print("\nnumber of Rows",df_customers.shape[0])\n",
    print("\nnumber of columns",df_customers.shape[1])\n"
  ]
},
{
  "cell_type": "markdown",
  "id": "5f228557-7f31-473e-b274-4d674547354e",
  "metadata": {},
  "source": [
    "# 5. get information about our data set like total number of rows,total number of columns,datatypes of each column and memory requirments\n"
  ]
},

```

```

{
  "cell_type": "code",
  "execution_count": null,
  "id": "11a069f9-d006-42bc-847f-19e4016388a7",
  "metadata": {},
  "outputs": [],
  "source": [
    "\n",
    "df_customers.info()"
  ]
},
{
  "cell_type": "markdown",
  "id": "aded8a24-d3f1-4258-bd61-7539c2b1c186",
  "metadata": {},
  "source": [
    "# 6. Check Null Values In the dataset\n"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "cf0a6fa7-0239-403f-8e3a-3c2c302e29b1",
  "metadata": {},
  "outputs": [],
  "source": [
    "df_customers.isnull().sum()\n"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "98c68c14-67b5-499b-897c-9ba25e538afe",
  "metadata": {},
  "outputs": [],
  "source": [
    "# Dropping the Nulls\n",
    "df = df_customers.dropna(subset=['TotalCharges'])\n",
    "df.isnull().sum()"
  ]
},
{
  "cell_type": "markdown",
  "id": "6fe3156b-4d4a-48e4-b05f-13d89ee072e7",
  "metadata": {},
  "source": [
    "# 7. Get overall Statistics About The Dataset"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "fe65f928-9f54-4bcd-b98d-1371aecbc978",
  "metadata": {},
  "outputs": [],
  "source": [
    "df.describe(include=\n"
  ]
},
{
  "cell_type": "markdown",
  "id": "109e8f3d-e28d-4a74-bdcb-702756a3e058",
  "metadata": {},
  "source": [
    "# 7. Checking Irrelevant features\n"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "98d1a83c-6bc9-4866-afef-1fcc85b878eb",
  "metadata": {},
  "outputs": [],
  "source": [
    "df.columns"
  ]
},
{
  "cell_type": "markdown",
  "id": "b52d778e-faa5-4368-955e-1797e9330168",
  "metadata": {},
  "source": [
    "# 8. Dropping Columns CustomerID ,PhoneService\n",
    "\n",
    "# Reasons \n",
    "CustomerID isJust an identifier, adds zero predictive value. DROP IT.\n",
    "\n",
    "PhoneService is redundant because if PhoneService = 'No', then MultipleLines is 'No phone service', so we already get that info. DROP ONE of the two. Pref
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "1b68317f-8139-4e86-95b7-03c4737aa471",
  "metadata": {},
  "outputs": [],
  "source": [
    "df = df.drop(['CustomerID', 'PhoneService'], axis=1)"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "ecfadba0-aa93-47db-af99-fbc20aca8c58",
  "metadata": {},
  "outputs": [],

```

```

"source": [
  "df.head()"
],
},
{
  "cell_type": "markdown",
  "id": "289b7e02-e45e-40a6-b515-ff0c2c064341",
  "metadata": {},
  "source": [
    "# 9.Encoding Categorical Data"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "a1a7516d-47ba-40a0-be2b-525be6dc4e4c",
  "metadata": {},
  "outputs": [],
  "source": [
    "#df1 is data after encoding\n",
    "df1 = pd.get_dummies(df, drop_first=True)\n"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "70fd2d2e-8ca0-45da-9354-5ea6fbb1f08a",
  "metadata": {},
  "outputs": [],
  "source": [
    "df1.head()"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "97ceblec-4a8d-46a1-a05f-7cf9fb99c3f1",
  "metadata": {},
  "outputs": [],
  "source": [
    "# Convert all boolean columns to integers (0/1)\n",
    "df1 = df1.astype(int)\n",
    "df1.head()"
  ]
},
{
  "cell_type": "markdown",
  "id": "89c54fff-b112-4f20-a814-0b01555f16dd",
  "metadata": {},
  "source": [
    "# 10. check that Data is BALanced Or not"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "729d7d78-b9de-46be-8136-81703380a508",
  "metadata": {},
  "outputs": [],
  "source": [
    "df1['Churn_Yes'].value_counts()"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "3e2384fb-4041-4080-a773-7c793f172c6e",
  "metadata": {},
  "outputs": [],
  "source": [
    "import seaborn as sns\n",
    "import matplotlib.pyplot as plt\n",
    "\n",
    "# Custom color palette: No = green, Yes = red\n",
    "sns.countplot(data=df, x='Churn', palette={'No': 'green', 'Yes': 'red'})\n",
    "\n",
    "plt.title(\"Churn Count\")\n",
    "plt.xlabel(\"Churn\")\n",
    "plt.ylabel(\"Number of Customers\")\n",
    "plt.show()"
  ]
},
{
  "cell_type": "markdown",
  "id": "831f7dab-935e-4c0e-b596-89b367201639",
  "metadata": {},
  "source": [
    "# Review \n",
    "its clearly shows that our data is imbalanced \n",
    "# solution \n",
    "we are going to use SMOTE for handing Imbalanced Data"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "919aa4ca-e5aa-49eb-acb7-926247589042",
  "metadata": {},
  "outputs": [],
  "source": [
    "#Step 1: Import Required Libraries for SMOTE and Model Training\n",
    "# For splitting the dataset\n",
    "from sklearn.model_selection import train_test_split\n",
    "\n",
    "# For oversampling\n",
    "from imblearn.over_sampling import SMOTE\n",

```

```

"\n",
"# For model training and evaluation\n",
"from sklearn.linear_model import LogisticRegression\n",
"from sklearn.metrics import classification_report, confusion_matrix\n"
],
{
"cell_type": "code",
"execution_count": null,
"id": "7884fa78-3070-4ac4-b1d8-9d4608a760c4",
"metadata": {},
"outputs": [],
"source": [
"# Step 2: Separate Features and Target\n",
"X = df1.drop('Churn_Yes', axis=1) # All columns except churn\n",
"y = df1['Churn_Yes']           # Target column: churn\n"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "c75c7217-f1a1-4ad3-95c8-28492eb73756",
"metadata": {
"scrolled": true
},
"outputs": [],
"source": [
"#Step 3: Split Data Into Train and Test Sets\n",
"X_train, X_test, y_train, y_test = train_test_split(\n",
"    X, y, test_size=0.2, random_state=42, stratify=y\n",
")"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "8e1fd3f1-b80e-4a49-b31c-9e2e9d818c26",
"metadata": {},
"outputs": [],
"source": [
"#Step 4: Apply SMOTE on Training Data ONLY\n",
"smote = SMOTE(random_state=42) # Create SMOTE object\n",
"\n",
"X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train) # Apply it\n"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "fd5c7ef8-2a95-4dcf-84d0-e52aef7e99e2",
"metadata": {},
"outputs": [],
"source": [
"import numpy as np\n",
"\n",
"unique, counts = np.unique(y_train_smote, return_counts=True)\n",
"for label, count in zip(unique, counts):\n",
"    print(f\"Class {label}: {count} samples\")"
]
},
{
"cell_type": "markdown",
"id": "dd2633e8-ecce-4931-ac13-0a5bf7f76546",
"metadata": {},
"source": [
"# 11. Applying Logisticregression model"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "bc96149b-4c9b-4e1b-a332-48537b86a7a6",
"metadata": {},
"outputs": [],
"source": [
"# 1. Instantiate the model\n",
"model = LogisticRegression(max_iter=1000, random_state=42)\n",
"\n",
"# 2. Fit on the SMOTE-balanced training set\n",
"model.fit(X_train_smote, y_train_smote)\n"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "dfa38df4-aa41-4a28-8ec9-8398676816d8",
"metadata": {},
"outputs": [],
"source": [
"# Predict churn on the untouched test data\n",
"y_pred = model.predict(X_test)\n"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "63a9d449-2e6d-4d6c-a852-f59482217470",
"metadata": {},
"outputs": [],
"source": [
"# 1. Confusion Matrix\n",
"cm = confusion_matrix(y_test, y_pred)\n",
"print(\"Confusion Matrix:\n\", cm)\n",
"\n",
"# 2. Classification Report (precision, recall, f1-score)\n",
"print(\"Classification Report:\n\", classification_report(y_test, y_pred))\n"
]
}

```

```

]
},
{
  "cell_type": "markdown",
  "id": "1ff8821d-cbcb-4557-bdb0-5c63e3fc5733",
  "metadata": {},
  "source": [
    "# Why This Happened (Even After SMOTE):\n",
    "SMOTE helped balance the training set, but:\n",
    "\n",
    "Logistic Regression is a linear model &€" may not capture complex churn patterns\n",
    "\n",
    "There might be non-linear relationships, interactions, or irrelevant features."
  ]
},
{
  "cell_type": "markdown",
  "id": "dcc34138-a42b-4a0d-9c16-67388914944e",
  "metadata": {},
  "source": [
    " # 12.Random Forest Classifier"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "12b6f5d1-7a35-4920-a3af-d1db895b1f1a",
  "metadata": {},
  "outputs": [],
  "source": [
    "from sklearn.ensemble import RandomForestClassifier\n",
    "from sklearn.metrics import confusion_matrix, classification_report\n",
    "\n",
    "# Initialize and train\n",
    "rf_model = RandomForestClassifier(random_state=42)\n",
    "rf_model.fit(X_train_smote, y_train_smote)\n",
    "\n",
    "# Predict\n",
    "y_pred_rf = rf_model.predict(X_test)\n",
    "\n",
    "# Evaluate\n",
    "print(\"ðŸŽ“ Random Forest Results:\")\n",
    "print(\"Confusion Matrix:\\n\", confusion_matrix(y_test, y_pred_rf))\n",
    "print(\"\\nClassification Report:\\n\", classification_report(y_test, y_pred_rf))\n"
  ]
},
{
  "cell_type": "markdown",
  "id": "e091ffc3-cf92-4502-89d3-61737e9153c4",
  "metadata": {},
  "source": [
    "# 13. Gradient Boosting Classifier"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "cee4a846-b828-4aff-9d88-e4a0868897d5",
  "metadata": {},
  "outputs": [],
  "source": [
    "from sklearn.ensemble import GradientBoostingClassifier\n",
    "\n",
    "# Initialize and train\n",
    "gb_model = GradientBoostingClassifier(random_state=42)\n",
    "gb_model.fit(X_train_smote, y_train_smote)\n",
    "\n",
    "# Predict\n",
    "y_pred_gb = gb_model.predict(X_test)\n",
    "\n",
    "# Evaluate\n",
    "print(\"ðŸŽ“ Gradient Boosting Results:\")\n",
    "print(\"Confusion Matrix:\\n\", confusion_matrix(y_test, y_pred_gb))\n",
    "print(\"\\nClassification Report:\\n\", classification_report(y_test, y_pred_gb))\n"
  ]
},
{
  "cell_type": "markdown",
  "id": "36d2772e-59cf-4404-8d31-68447398c787",
  "metadata": {},
  "source": [
    "# 14.Support Vector Machine (SVM)"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "770e613c-2c11-4365-8ab8-8ae95349814d",
  "metadata": {},
  "outputs": [],
  "source": [
    "from sklearn.svm import SVC\n",
    "\n",
    "# Initialize and train\n",
    "svc_model = SVC(probability=True, random_state=42)\n",
    "svc_model.fit(X_train_smote, y_train_smote)\n",
    "\n",
    "# Predict\n",
    "y_pred_svc = svc_model.predict(X_test)\n",
    "\n",
    "# Evaluate\n",
    "print(\"ðŸŽ“ SVM Results:\")\n",
    "print(\"Confusion Matrix:\\n\", confusion_matrix(y_test, y_pred_svc))\n",
    "print(\"\\nClassification Report:\\n\", classification_report(y_test, y_pred_svc))\n"
  ]
},

```

```

{
  "cell_type": "markdown",
  "id": "455e2a47-72ce-4eb3-8100-e3409f51bff7",
  "metadata": {},
  "source": [
    "# 15. Try XGBoost"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "58ae049c-05ca-4bb5-9912-1d006e21ba87",
  "metadata": {},
  "outputs": [],
  "source": [
    "from xgboost import XGBClassifier\n",
    "\n",
    "xgb_model = XGBClassifier(random_state=42, use_label_encoder=False, eval_metric='logloss')\n",
    "xgb_model.fit(X_train_smote, y_train_smote)\n",
    "\n",
    "y_pred_xgb = xgb_model.predict(X_test)\n",
    "\n",
    "print("\nXGBoost Results:\n")\n",
    "print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_xgb))\n",
    "print("\nClassification Report:\n", classification_report(y_test, y_pred_xgb))\n"
  ]
},
{
  "cell_type": "markdown",
  "id": "3b1256cb-0671-41ec-8115-7df1ad02b11f",
  "metadata": {},
  "source": [
    "# ROC Curve + AUC Score"
  ]
},
{
  "cell_type": "markdown",
  "id": "5687c6de-93f5-450c-a355-a0ed72840e2d",
  "metadata": {},
  "source": [
    "### For Logistic Regression model's performance"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "be4279f7-f692-4484-bba8-ea255de1fc47",
  "metadata": {},
  "outputs": [],
  "source": [
    "from sklearn.metrics import roc_curve, auc\n",
    "import matplotlib.pyplot as plt\n",
    "\n",
    "# Get predicted probabilities for the positive class (churn = 1)\n",
    "y_probs = model.predict_proba(X_test)[:, 1]\n",
    "\n",
    "# Calculate ROC curve values\n",
    "fpr, tpr, thresholds = roc_curve(y_test, y_probs)\n",
    "\n",
    "# Calculate AUC score\n",
    "roc_auc = auc(fpr, tpr)\n",
    "print(f"ROC AUC Score: {roc_auc:.4f}")\n",
    "\n",
    "# Plot the ROC curve\n",
    "plt.figure(figsize=(8,6))\n",
    "plt.plot(fpr, tpr, color='blue', label=f'ROC curve (AUC = {roc_auc:.4f})')\n",
    "plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Guess')\n",
    "plt.xlabel('False Positive Rate')\n",
    "plt.ylabel('True Positive Rate')\n",
    "plt.title('ROC Curve for Logistic Regression')\n",
    "plt.legend(loc='lower right')\n",
    "plt.grid(True)\n",
    "plt.show()\n"
  ]
},
{
  "cell_type": "markdown",
  "id": "4ac8b1e9-095a-428e-b9ac-41b3e42ee981",
  "metadata": {},
  "source": [
    "# Random Forest model"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "8a95e6eb-90e3-40e4-a6a0-6026516ddd6f",
  "metadata": {},
  "outputs": [],
  "source": [
    "from sklearn.metrics import roc_curve, auc\n",
    "import matplotlib.pyplot as plt\n",
    "\n",
    "# Get predicted probabilities for positive class (churn=1)\n",
    "y_probs_rf = rf_model.predict_proba(X_test)[:, 1]\n",
    "\n",
    "# Calculate ROC curve values\n",
    "fpr_rf, tpr_rf, thresholds_rf = roc_curve(y_test, y_probs_rf)\n",
    "\n",
    "# Calculate AUC score\n",
    "roc_auc_rf = auc(fpr_rf, tpr_rf)\n",
    "print(f"Random Forest ROC AUC Score: {roc_auc_rf:.4f}")\n",
    "\n",
    "# Plot the ROC curve\n",
    "plt.figure(figsize=(8,6))\n",
    "plt.plot(fpr_rf, tpr_rf, color='green', label=f'ROC curve (AUC = {roc_auc_rf:.4f})')\n"
  ]
}

```

```

"plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Guess')\n",
"plt.xlabel('False Positive Rate')\n",
"plt.ylabel('True Positive Rate')\n",
"plt.title('ROC Curve for Random Forest Classifier')\n",
"plt.legend(loc='lower right')\n",
"plt.grid(True)\n",
"plt.show()\n"
]
},
{
"cell_type": "markdown",
"id": "3042a5bb-6872-4e9e-b225-b5465e85f860",
"metadata": {},
"source": [
"# Gradient Boosting Classifier"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "3a83265d-f18b-40bd-9332-186d825ff3a3",
"metadata": {},
"outputs": [],
"source": [
"from sklearn.metrics import roc_curve, auc\n",
"import matplotlib.pyplot as plt\n",
"\n",
"# Get predicted probabilities for positive class (churn=1)\n",
"y_probs_gb = gb_model.predict_proba(X_test)[:, 1]\n",
"\n",
"# Calculate ROC curve values\n",
"fpr_gb, tpr_gb, thresholds_gb = roc_curve(y_test, y_probs_gb)\n",
"\n",
"# Calculate AUC score\n",
"roc_auc_gb = auc(fpr_gb, tpr_gb)\n",
"print(f\"Gradient Boosting ROC AUC Score: {roc_auc_gb:.4f}\")\n",
"\n",
"# Plot the ROC curve\n",
"plt.figure(figsize=(8,6))\n",
"plt.plot(fpr_gb, tpr_gb, color='blue', label=f'ROC curve (AUC = {roc_auc_gb:.4f})')\n",
"plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Guess')\n",
"plt.xlabel('False Positive Rate')\n",
"plt.ylabel('True Positive Rate')\n",
"plt.title('ROC Curve for Gradient Boosting Classifier')\n",
"plt.legend(loc='lower right')\n",
"plt.grid(True)\n",
"plt.show()\n"
]
},
{
"cell_type": "markdown",
"id": "1858831a-072f-4972-bda6-7ff1411b200f",
"metadata": {},
"source": [
"# Support Vector Machine (SVM)"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "05baa4b2-8fb1-4d94-b7c7-a85d26f39f59",
"metadata": {},
"outputs": [],
"source": [
"from sklearn.metrics import roc_curve, auc\n",
"import matplotlib.pyplot as plt\n",
"\n",
"# Get predicted probabilities for positive class (churn=1)\n",
"y_probs_svc = svc_model.predict_proba(X_test)[:, 1]\n",
"\n",
"# Calculate ROC curve values\n",
"fpr_svc, tpr_svc, thresholds_svc = roc_curve(y_test, y_probs_svc)\n",
"\n",
"# Calculate AUC score\n",
"roc_auc_svc = auc(fpr_svc, tpr_svc)\n",
"print(f\"SVM ROC AUC Score: {roc_auc_svc:.4f}\")\n",
"\n",
"# Plot the ROC curve\n",
"plt.figure(figsize=(8,6))\n",
"plt.plot(fpr_svc, tpr_svc, color='purple', label=f'ROC curve (AUC = {roc_auc_svc:.4f})')\n",
"plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Guess')\n",
"plt.xlabel('False Positive Rate')\n",
"plt.ylabel('True Positive Rate')\n",
"plt.title('ROC Curve for SVM Classifier')\n",
"plt.legend(loc='lower right')\n",
"plt.grid(True)\n",
"plt.show()\n"
]
},
{
"cell_type": "markdown",
"id": "ffa4e5b2-628f-437e-9c56-97f084ecfb92",
"metadata": {},
"source": [
"# XGBoost"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "c879a95d-8f70-449e-99de-d51a8621b952",
"metadata": {},
"outputs": [],
"source": [
"from sklearn.metrics import roc_curve, auc\n",
"import matplotlib.pyplot as plt\n",

```



```

"\n",
"# Predict probabilities for positive class (churn=1)\n",
"y_probs_xgb = xgb_model.predict_proba(X_test)[: , 1]\n",
"\n",
"# Calculate ROC curve\n",
"fpr_xgb, tpr_xgb, _ = roc_curve(y_test, y_probs_xgb)\n",
"\n",
"# Calculate AUC\n",
"roc_auc_xgb = auc(fpr_xgb, tpr_xgb)\n",
"print(f\"XGBoost ROC AUC Score: {roc_auc_xgb:.4f}\")\n",
"\n",
"# Plot ROC curve\n",
"plt.figure(figsize=(8,6))\n",
"plt.plot(fpr_xgb, tpr_xgb, color='orange', label=f'ROC curve (AUC = {roc_auc_xgb:.4f})')\n",
"plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random Guess')\n",
"plt.xlabel('False Positive Rate')\n",
"plt.ylabel('True Positive Rate')\n",
"plt.title('ROC Curve for XGBoost Classifier')\n",
"plt.legend(loc='lower right')\n",
"plt.grid(True)\n",
"plt.show()\n"
]
},
{
"cell_type": "markdown",
"id": "f8905e8b-a839-43ac-96ed-a0ee3357e67a",
"metadata": {},
"source": [
"# Model Comparison Gather all Our scores and ROC-AUC"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "0247187c-cf2a-415f-a710-a01e0ec943be",
"metadata": {},
"outputs": [],
"source": [
"from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score\n",
"\n",
"models = {\n",
"    'Logistic Regression': model,\n",
"    'Random Forest': rf_model,\n",
"    'Gradient Boosting': gb_model,\n",
"    'SVM': svc_model\n",
"}\n",
"\n",
"results = []\n",
"for name, mdl in models.items():\n",
"    y_pred = mdl.predict(X_test)\n",
"    y_proba = mdl.predict_proba(X_test)[: , 1] # For ROC AUC\n",
"    \n",
"    results.append({\n",
"        'Model': name,\n",
"        'Accuracy': accuracy_score(y_test, y_pred),\n",
"        'Precision': precision_score(y_test, y_pred),\n",
"        'Recall': recall_score(y_test, y_pred),\n",
"        'F1 Score': f1_score(y_test, y_pred),\n",
"        'ROC AUC': roc_auc_score(y_test, y_proba)\n",
"    })\n",
"\n",
"import pandas as pd\n",
"results_df = pd.DataFrame(results)\n",
"print(results_df.sort_values(by='ROC AUC', ascending=False))\n"
]
},
{
"cell_type": "markdown",
"id": "400f4af7-9f63-434d-9e3a-6647bc51eedc",
"metadata": {},
"source": [
"# Hyperparameter tuning with GridSearchCV for Logistic Regression and Random Forest"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "45217e9f-7251-4e45-9cb9-196429bd867f",
"metadata": {},
"outputs": [],
"source": [
"from sklearn.model_selection import GridSearchCV\n",
"\n",
"# Logistic Regression hyperparameter grid\n",
"param_grid_lr = {\n",
"    'C': [0.01, 0.1, 1, 10],\n",
"    'penalty': ['l2'], # L1 might cause convergence issues, but you can try it later\n",
"    'solver': ['lbfgs', 'saga']\n",
"}\n",
"\n",
"grid_lr = GridSearchCV(LogisticRegression(max_iter=1000, random_state=42), param_grid_lr, cv=5, scoring='roc_auc')\n",
"grid_lr.fit(X_train_smote, y_train_smote)\n",
"\n",
"print(\"Best Logistic Regression Params:\", grid_lr.best_params_)\n",
"print(\"Best Logistic Regression ROC AUC:\", grid_lr.best_score_)\n"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "a9e2be07-25ac-4053-9b8f-56fb7ab65ef5",
"metadata": {},
"outputs": [],
"source": [
"# Random Forest hyperparameter grid\n",

```

```

"param_grid_rf = {\n",
"    'n_estimators': [100, 200],\n",
"    'max_depth': [5, 10, None],\n",
"    'min_samples_split': [2, 5],\n",
"    'min_samples_leaf': [1, 2]\n",
"}\n",
"\n",
"grid_rf = GridSearchCV(RandomForestClassifier(random_state=42), param_grid_rf, cv=5, scoring='roc_auc', n_jobs=-1)\n",
"grid_rf.fit(X_train_smote, y_train_smote)\n",
"\n",
"print("\nBest Random Forest Params:\n", grid_rf.best_params_)\n",
"print("\nBest Random Forest ROC AUC:\n", grid_rf.best_score_)\n",
}
},
{
"cell_type": "markdown",
"id": "dbd53b80-a612-4ec9-b3c4-ce71e9c6274b",
"metadata": {},
"source": [
    "# This means our model's now wayyy better at distinguishing churners from non-churners."
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "ec856051-1868-4abe-b225-aea40084451c",
"metadata": {},
"outputs": [],
"source": [
    "# Retrain best RF model\n",
    "best_rf = RandomForestClassifier(\n",
    "    max_depth=10,\n",
    "    min_samples_leaf=1,\n",
    "    min_samples_split=2,\n",
    "    n_estimators=100,\n",
    "    random_state=42\n",
    ")\n",
    "\n",
    "best_rf.fit(X_train_smote, y_train_smote)\n",
    "\n",
    "# Predict on test data\n",
    "y_pred_best_rf = best_rf.predict(X_test)\n",
    "y_pred_prob_rf = best_rf.predict_proba(X_test)[:, 1]\n",
    "\n",
    "# Metrics\n",
    "from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, classification_report\n",
    "\n",
    "print("\nðŸŽ“ Tuned Random Forest Test Results:\n")\n",
    "print("\nAccuracy:\n", accuracy_score(y_test, y_pred_best_rf))\n",
    "print("\nPrecision:\n", precision_score(y_test, y_pred_best_rf))\n",
    "print("\nRecall:\n", recall_score(y_test, y_pred_best_rf))\n",
    "print("\nF1 Score:\n", f1_score(y_test, y_pred_best_rf))\n",
    "print("\nROC AUC:\n", roc_auc_score(y_test, y_pred_prob_rf))\n",
    "print("\n\\nConfusion Matrix:\\n", confusion_matrix(y_test, y_pred_best_rf))\n",
    "print("\n\\nClassification Report:\\n", classification_report(y_test, y_pred_best_rf))\n",
]
},
{
"cell_type": "markdown",
"id": "fb88b995-909f-4eba-8ad2-06000dd6b7ac",
"metadata": {},
"source": [
    "## ðŸŽ“ Summary & Key Takeaways\n",
    "\n",
    "- The tuned Random Forest model achieved a test ROC AUC of **0.569** and accuracy of **63%**.\n",
    "- It performs better at predicting the majority class (non-churn), but struggles with minority class (churn), as seen from lower recall.\n",
    "- Top features (from feature importance) include: `tenure`, `MonthlyCharges`, and `Contract` type.\n",
    "- Future improvements could involve:\n",
    "    - Trying more advanced models like XGBoost or LightGBM.\n",
    "    - Further feature engineering or dimensionality reduction.\n",
    "    - Collecting more data or using cost-sensitive learning techniques.\n",
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "b472dd56-6664-488e-bd6c-f475efab38c7",
"metadata": {},
"outputs": [],
"source": [
    "import joblib\n",
    "joblib.dump(svc_model, \"tuned_random_forest.pkl\") \n",
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "d644d5d2-4b5a-421f-938c-2958909a71c4",
"metadata": {},
"outputs": [],
"source": []
}
},
"metadata": {
"kernel_spec": {
"display_name": "Python 3 (ipykernel)",
"language": "python",
"name": "python3"
},
"language_info": {
"codemirror_mode": {
"name": "ipython",
"version": 3
},
"file_extension": ".py",
"mimetype": "text/x-python",

```

```
"name": "python",
"nbconvert_exporter": "python",
"pygments_lexer": "ipython3",
"version": "3.13.0"
},
"nbformat": 4,
"nbformat_minor": 5
}
```